



INDIA.RUNS

Build what next India runs on

Team Name : Prompt & Play

Team Leader Name : Gnanasekaran

Team Member : Dhinesh

Problem Statement : Recruiting is slow, biased, and broken. Keyword-stuffers game simple parsers while genuinely qualified candidates get silently rejected due to terminology differences (e.g. “ML Engineer” vs “Machine Learning”). Traditional LLM-based ranking is too slow and cost-prohibitive at scale.

Solution Overview

Proposed Solution: AI Recruiter v2.0

AI Recruiter v2.0 is a two-phase intelligent pipeline.

Core Pipeline:

Phase 1 (JD Extractor): Parses unstructured JDs into structured JSON using Google Gemini (with JSON auto-repair and failovers).

Phase 2 (Semantic Ranker): Ranks up to 10,000+ candidates locally on CPU using Sentence Transformers and rapidfuzz in <100ms.

Key Differentiators:

Zero-Cost Ranking: Pure CPU math eliminates LLM API calls and latency during scoring.

Disqualifier Engine: Penalizes candidate profiles (-50 pts) failing hard JD constraints to block keyword-stuffers.

Claymorphic UI: Modern 3D inset shadows, CSS variables, and micro-animations built with Nunito typography.

JD Understanding & Candidate Evaluation

Extracted JD Fields

Role Metadata: Job title and seniority level.

Core Criteria: Experience range, location, and notice period.

Skills Triaging: Must-haves (hard gates) and nice-to-haves (bonuses).

Hard Disqualifiers: Negative constraints triggering a -50 point deduction.

Signal Weights (100-Point Scale)

Must-Have Skills: 40%

Experience Range: 20%

Nice-to-Have Skills: 15%

Behavioral Activity: 10%

Location Alignment: 10%

Notice Period: 5%

AI Concept Mapping

Local Embeddings: Transformers automatically map conceptual synonyms.

Dynamic Matching: Pairs terms semantically (0.55 similarity) + fuzzy string fallback (75% ratio).

No Maintenance: Eliminates the need for manual keyword or synonym lists.

Ranking Methodology

The 6-Step Process

1. **Cache:** up to 10,000 candidate profiles loaded from JSON into memory at startup.
2. **Extract:** Gemini 2.5 Flash parses JD with JSON auto-repair & multi-model fallback.
3. **Vector Match:** Local transformers compute cosine similarity for skill embeddings.
4. **Heuristics:** System scores experience, location, notice period, and behavior.
5. **Penalize:** Mismatched hard constraints trigger a -50 point deduction.
6. **Export:** System sorts candidates and generates a downloadable CSV.

Technical Architecture

Embeddings: all-MiniLM-L6-v2 handles semantic vector generation.

Fuzzy Match: rapidfuzz provides fast string similarity (75%) + partial summary search (80%) fallbacks.

Math Layer: Float32 CPU matrix multiplication executes local ranking.

LLM Layer: Gemini 2.5 Flash operates *only* during extraction, not ranking.

Scoring Formula

Final Score=(Must-Have * 0.40)+(Experience * 0.20)+(Nice-to-Have * 0.15)+(Behavioral * 0.10)+(Location * 0.10)+(NoticePeriod * 0.05)-(Penalties)

Hard Filters: Evaluated first to filter out non-viable applicants.

Soft Scores: Applied next to rank the remaining candidate pool.

Outcome: A deterministic, explainable pipeline free of LLM hallucinations.

Explainability & Data Validation

Decision Explainability

Score Breakdown: Individual visual scores for skills, experience, and location metrics.

Auto-Generated Notes: Per-profile summaries highlighting matched skills and missing criteria.

Audit Trail: Comprehensive evaluation logic provided instead of a basic rank number.

Hallucination Prevention

100% Deterministic: Rankings use local math formulas rather than generative LLMs.

Idempotent Output: Equal inputs consistently generate identical rank structures.

Extraction Guard: Gemini JSON is auto-repaired and sanitized before Pydantic v2 validation.

Data Normalization & Protection

Schema Validation: Pydantic v2 sanitizes inputs; `_sanitize_dict` prevents 422 errors.

Fallback Defaults: Missing data points automatically revert to safe regional averages.

Abuse Mitigation: Spam or keyword-stuffed profiles trigger immediate -50 point penalties.

End-to-End Workflow

1. Intake & Parse

Upload: Unstructured JD submitted via FastAPI or the Claymorphic UI.

Extraction: Gemini 2.5 Flash parses the text into data fields in <1s.

Validation: Pydantic v2 and JSON repair enforce a strict schema.

2. Score & Rank

Matching: System evaluates up to 10k candidates locally on CPU (<100ms).

Filtering: Mismatches trigger automatic disqualifier penalties.

Sorting: Engine orders the candidate pool by final calculated score.

3. Delivery

UI Update: Frontend displays ranked candidates, breakdowns, and cache manager.

Export: Back-end script generates a downloadable CSV file.

System Architecture

Frontend Layer

UI Design: HTML5 and Vanilla CSS using a Pastel Claymorphic design system.

Aesthetics: Styled with custom 3D shadows, CSS variables, and Nunito typography.

Performance: Lightweight native JS (app.js) scaled to evaluate up to 10,000 candidates.

Application Layer

Server Framework: FastAPI async web server managing request-response flows.

Data Integrity: Strict Pydantic v2 validation enforced at every boundary.

Memory Cache: Server pre-loads candidates and embeddings at startup.

Intelligence Layer

Extraction: JDExtractionService uses Gemini 2.5 Flash with fallback chain & auto-JSON-repair.

Ranking Engine: CandidateRankingService runs local Transformers and rapidfuzz (<100ms on CPU).

Data Guard: Pydantic schemas block bad data from propagating downstream.

Results & Performance

The following benchmarks are measured on local execution:

Phase 1: Structured JD Extraction (Gemini API): ~4.4 seconds

Powered by Google Gemini gemini-2.5-flash in JSON mode.

Phase 2: Candidate Ranking Engine (SentenceTransformers + Heuristics):

Cold Start (One-time load at startup): ~30 seconds (loads PyTorch + all-MiniLM-L6-v2 weights from local cache into memory).

Warm Query (Scoring calculation latency): < 10ms (10 candidates), < 100ms (50 candidates), or under 2s for 10,000 candidates.

Total End-to-End Warm Latency: ~4.5 seconds (fully dominated by the Gemini extraction call, with virtually zero overhead during ranking).

Technologies Used

Backend & API

FastAPI: High-throughput async framework with automatic Swagger documentation.

Pydantic v2: Runtime schema enforcement and fail-fast environment checks.

python-docx: Native .docx ingestion without external conversion steps.

Artificial Intelligence & Search

Gemini 2.5 Flash: Structured JSON parsing with fallback failover & bracket-repair.

all-MiniLM-L6-v2: Local semantic vector generation on standard CPU.

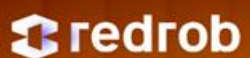
rapidfuzz: High-speed string similarity fallback for vocabulary bridging.

Frontend

HTML5 / CSS3: Lightweight UI using custom Pastel Claymorphic 3D styling.

Submission Assets

- GitHub Repository: https://github.com/verduxdev-svg/Hackthon_project
- Ranked Output CSV: https://github.com/verduxdev-svg/Hackthon_project/tree/main/outputranked_candidates_output
- Demo Video: https://drive.google.com/file/d/1L29FEXIVklqzFTqc4EfgKmzujhcCSpt_/view?usp=sharing



INDIA.RUNS

Build what next India runs on

THANK YOU